

Test-Time Search Degrades Control Policies When the Verifier Is Learned

Andrew Haggstrom

August 12, 2026

Abstract

Scaling inference-time computation has produced large gains in language models, and recent work reports similar gains for robot policies by sampling multiple candidate actions and selecting among them with a learned verifier. We evaluate this procedure across six Meta-World manipulation tasks at candidate counts up to $K = 64$ and find that it consistently *harms* performance: pooled success falls from 0.62 at $K = 1$ to 0.40 at $K = 64$, below the 0.58 achieved by selecting candidates uniformly at random. Exploiting exact simulator state restoration, we show that this is not a limitation of search itself—an oracle verifier using true dynamics improves success on three of six tasks—but of the learned verifier, whose selections are worth no more than an arbitrary candidate even as substantially better candidates become available. Penalizing ensemble disagreement does not help. Verifier-free selection by candidate consensus does not degrade. We argue that the harm accumulates from small per-decision errors across many decisions, which may explain why prior evaluations at smaller K and shorter horizons did not observe it.

1 Introduction

Language models have improved substantially by spending more computation at inference time rather than by growing larger: sampling many candidate solutions and selecting among them, reasoning step by step, or verifying and retrying [Snell et al., 2024]. The underlying weights are unchanged; what changes is how much work the model does per query.

Control policies do not generally do this. A visuomotor or state-based policy observes, produces an action, and executes it, typically within a fixed budget of one forward pass. There is no established analogue of “thinking longer” about a difficult decision.

The natural port is straightforward to state. A stochastic policy can sample K candidate actions or action chunks; a scoring function ranks them; the highest-scoring candidate is executed. The candidate count K becomes an inference-time compute knob directly analogous to best-of- N sampling in language models. Several recent systems implement approximately this and report substantial gains [RoboMonkey, 2025, RoVer, 2025, ELASTIC, 2026, CoVer-VLA, 2026].

The difficulty is the scoring function. Selecting the best candidate requires knowing which candidate is best, which is precisely the knowledge that imitation learning does not provide. In practice the score comes from a learned dynamics model, a learned value function, or a learned reward model—each an imperfect proxy for the quantity of interest. Optimizing against an imperfect proxy is exactly the setting in which Goodhart’s law applies: as selection pressure increases, the argmax increasingly identifies the candidate for which the proxy is most *wrong* rather than the candidate that is genuinely best. This phenomenon is well documented for reward models in language modeling [Gao et al., 2022], where performance rises with search budget, peaks, and then declines.

Whether the same holds for embodied control has not been characterized. The robotics test-time-scaling literature reports monotone improvement, but typically evaluates $K \leq 32$ and stops while curves are still rising. Two features of control make the question sharper than in the language setting. First, an episode contains many decisions—in our tasks, 15 to 45—so per-decision errors compound in a way that single-shot generation does not. Second, unlike a language model’s reward model, a dynamics model’s error is measurable against ground truth, which makes the mechanism accessible.

Simulation offers a methodological advantage that physical robots cannot. MuJoCo permits exact save and restore of simulator state, so a candidate can be evaluated by executing it in the true environment and rewinding. This yields an *oracle verifier*: search with a perfect model. The oracle arm is what makes negative results interpretable. Without it, a flat or declining curve is ambiguous between three explanations—the candidate set contains nothing better, the search procedure is broken, or the verifier is inadequate. With it, these separate cleanly.

We evaluate nine selection strategies on six Meta-World V3 manipulation tasks, sweeping K from 1 to 64 with 50 episodes per condition, paired across strategies by episode seed. Our findings are:

1. **Learned-verifier search is worse than not searching, and worse than random selection.** Pooled success falls from 0.62 to 0.40 as K grows to 64, while random selection among the same candidates remains near 0.58. The degradation is monotone from $K = 2$; there is no beneficial regime.
2. **Better candidates exist; the verifier cannot find them.** An oracle verifier improves success on three of six tasks. A direct audit shows that as K grows the best available candidate improves substantially while the learned verifier’s selection remains statistically indistinguishable from an arbitrary candidate.
3. **Uncertainty penalties do not help.** Penalizing ensemble disagreement, with penalty strength calibrated per task, leaves the degradation essentially unchanged across two orders of magnitude.
4. **Verifier-free selection is immune.** Selecting the medoid of the candidate set—consensus, with no learned scorer—does not degrade at any K . Having no proxy, it has no proxy to over-optimize.

We also report a null result on the relationship between dynamics-model accuracy and degradation, and a weaker hypothesis about objective misspecification on one task where even the oracle verifier fails.

2 Related work

Test-time scaling in language models. Allocating additional computation at inference has been shown to be a favorable trade against additional parameters under some conditions [Snell et al., 2024]. The dominant mechanisms are parallel sampling with verifier-based selection, sequential revision, and search over intermediate reasoning states. Our formulation of candidate sampling with verifier selection is the direct control analogue of best-of- N .

Reward-model over-optimization. gao2022scaling characterize how performance against a true objective degrades as optimization pressure against a learned proxy reward increases, establishing the inverted-U shape that motivates our study. The mitigation literature emphasizes regularization—KL penalties, uncertainty penalties, or early stopping in the search parameter—rather than stronger search. Our disagreement-penalty experiments test the control analogue of this remedy.

Test-time scaling for robot policies. RoboMonkey [RoboMonkey, 2025] samples actions from a vision-language-action policy, constructs a proposal distribution via perturbation and majority voting, and selects using a VLM-based verifier, reporting substantial gains on out-of-distribution tasks. RoVer [RoVer, 2025] expands candidates along directions suggested by a process reward model. ELASTIC [ELASTIC, 2026] learns to allocate adaptively between parallel sampling and sequential refinement. CoVer-VLA [CoVer-VLA, 2026] characterizes scaling behavior for instruction following. All report improvement with additional candidates. Our work differs in three respects: we evaluate to $K = 64$ rather than stopping in the tens; we use tasks with 15–45 decisions per episode, so per-decision effects accumulate; and we include an oracle-verifier arm that bounds what search could achieve with a perfect model.

Model-based planning and horizon. Planning against learned dynamics has a long history in control. PETS [Chua et al., 2018] established probabilistic ensembles with trajectory sampling as a strong model-based method, and our dynamics model follows its design. MBPO [Janner et al., 2019] formalizes the tradeoff between planning horizon and compounding model error, which motivates our horizon calibration. TD-MPC2 [Hansen et al., 2023] represents the current state of the art in learned-model planning. Our learned verifier is a deliberately standard instance of this family rather than a novel planner; the question is what happens as selection pressure over its scores increases.

Objective mismatch. lambert2020objective observe that dynamics-model accuracy and downstream control performance are only loosely coupled: a model can improve on its training objective while control degrades. Our observation that a perfect dynamics model still fails on one task, and that model error across a $12\times$ range shows no relationship to degradation, is consistent with this line of work and suggests the scoring objective—not the model—is sometimes the binding constraint.

Action-chunked imitation learning. Predicting short action sequences rather than single actions is now standard [Chi et al., 2023, Zhao et al., 2023]. We adopt chunked prediction with a fixed execution stride, which defines the decision points at which search occurs.

3 Methods

3.1 Tasks and demonstrations

We use six Meta-World V3 manipulation tasks, selected from a screening pass over the full suite on two criteria: the scripted expert must succeed reliably enough to provide clean demonstrations, and the task horizon must be long enough that an episode contains many decision points. Table 1 summarizes the suite. We deliberately excluded short tasks such as reach and push, on which scripted experts finish in under 60 steps, because at an execution stride of 8 they afford fewer than 8 selection decisions per episode.

Task	Expert success	Demo length	Base policy	Decisions/episode
peg-insert-side	0.88	145	0.60	~35
basketball	0.91	132	0.45	~40
stick-pull	0.95	143	0.65	~32
shelf-place	0.96	114	0.65	~31
disassemble	0.98	135	0.65	~31
bin-picking	1.00	125	0.40	~45

Table 1: Task suite. Expert success is measured under the scripted policy used for demonstration collection; base policy success is the cloned policy at $K = 1$.

We collect 100 demonstrations per task from Meta-World’s scripted policies. We found that injecting Gaussian noise into expert actions—a common method for degrading demonstration quality—is ineffective here: the scripted policies are closed-loop and correct for injected noise within a few steps, so success rates are nearly unchanged even at $\sigma = 0.5$. Episodes terminate at first success rather than running to the time limit, so demonstration length reflects task difficulty and post-success steps are not recorded as expert behavior.

3.2 Base policy

For each task we train a separate policy π_θ mapping the 39-dimensional Meta-World state to a 16-step action chunk $a_{1:H} \in [-1, 1]^{16 \times 4}$, using a three-layer MLP with 512 hidden units and a tanh output, trained under mean squared error. Candidates are generated by adding isotropic Gaussian noise to the predicted chunk:

$$a_{1:H}^{(i)} = \text{clip} \left(\pi_\theta(s) + \epsilon^{(i)}, -1, 1 \right), \quad \epsilon^{(i)} \sim \mathcal{N}(0, \sigma_s^2 I), \quad \sigma_s = 0.15. \quad (1)$$

We target intermediate base competence: a policy near ceiling leaves no headroom for search to demonstrate benefit, and a policy near zero cannot be rescued by selection. The resulting policies achieve 0.40–0.65 success (Table 1), which arises from task difficulty at 100 demonstrations rather than from any deliberate degradation.

We note that this candidate distribution is unimodal by construction. This matches the proposal mechanism used by RoboMonkey [RoboMonkey, 2025] but differs from sampling a genuinely multimodal generative policy, and we return to this in the limitations.

3.3 Dynamics ensemble

The learned verifier uses a probabilistic ensemble of $M = 5$ networks in the style of PETS [Chua et al., 2018]. Each member predicts a state delta and reward with learned heteroscedastic variance, trained under Gaussian negative log-likelihood:

$$\mathcal{L}_m = E \left[(\mu_m(s, a) - y)^\top \Sigma_m(s, a)^{-1} (\mu_m(s, a) - y) + \log \det \Sigma_m(s, a) \right], \quad y = [s' - s, r]. \quad (2)$$

Members are trained on independently bootstrapped batches so that inter-member disagreement reflects epistemic uncertainty rather than initialization noise. We define disagreement at a state as the mean per-dimension standard deviation across members, normalized by the data standard deviation.

Two implementation details materially affect correctness. Meta-World’s observation vector contains dimensions that are constant across the dataset; normalizing these by their near-zero

standard deviation amplifies small predicted deltas by roughly 10^6 per step, which causes multi-step rollouts to diverge and contaminates the disagreement statistic. We mask constant dimensions from both the delta update and the disagreement computation, and raise the normalization floor at inference.

3.4 Value function

A separate MLP $V_\phi(s)$ is trained on Monte Carlo discounted returns ($\gamma = 0.99$) computed from the demonstration data. It serves both as a terminal bootstrap for learned-dynamics rollouts and, in the value best-of- N arm, as a standalone scorer.

3.5 Planning horizon

The learned verifier rolls candidates forward for h steps and bootstraps the remainder with V_ϕ . We select h by measuring prediction error against ground-truth simulator rollouts. Median error, expressed in units of the observation standard deviation, grows from 0.34σ at $h = 1$ to 0.71σ at $h = 2$, 1.52σ at $h = 4$, 3.75σ at $h = 8$, and 6.91σ at $h = 16$. We set $h = 4$, the longest horizon under a 2σ threshold. Planning the full 16-step chunk would score candidates in a regime where the model has already lost track of the state.

3.6 Selection arms

At each decision point the policy samples K candidates and a selector returns the index to execute. All selectors implement the same interface, so K and the selection rule vary independently.

No search. Executes the first sample; equivalent to $K = 1$ regardless of how many candidates were drawn. This arm must be invariant to K , which we use as a correctness check on episode pairing.

Random. Selects uniformly at random. This isolates the effect of *selecting on predicted value* from the effect of drawing more candidates.

Oracle. Scores each candidate by executing it in the true simulator and restoring state afterward: $\sum_t \gamma^t r_t + \gamma^t \mathbf{1}[\text{success}] \cdot c$.

Learned dynamics. Scores by rolling the ensemble forward $h = 4$ steps:

$$\text{score}(a_{1:H}) = \sum_{t=1}^h \gamma^t \hat{r}_t + \gamma^h V_\phi(\hat{s}_h) - \beta d(a_{1:H}), \quad (3)$$

where d is mean ensemble disagreement over the rollout and β is the uncertainty-penalty strength.

Value best-of- N . Scores by V_ϕ applied to the predicted one-step-ahead state, the cheapest verifier and the closest analogue to reward-model best-of- N .

Self-consistency. Selects the medoid of the candidate set under a weighted L_2 metric over chunks. Uses no learned scorer.

Calibrating β . The penalty term is only meaningful if it is commensurate with the return term. We measure both at real decision points: on stick-pull at $h = 4$, the median spread of predicted returns across 64 candidates is 0.654 while the median disagreement spread is 0.026, a ratio of roughly 25. We therefore sweep $\beta \in \{0, 0.25\rho, \rho, 4\rho\}$ where ρ is that per-task ratio, spanning no penalty through a dominant one. Values chosen without this calibration—for instance $\beta \in \{0, 1, 5\}$ —would have been 20 to 400 $\times$ too small to affect selection on some tasks.

3.7 Evaluation protocol

We sweep $K \in \{1, 2, 4, 8, 16, 32, 64\}$ with 50 episodes per cell. Two design choices reduce variance substantially. First, candidates are sampled once at $K_{\max}$ per decision point and smaller K evaluate nested prefixes, so curves across K are perfectly paired. Second, the environment is constructed with the episode index as its seed, so every selector faces the identical task instance on episode i . Meta-World draws goal configurations at construction time; without seeding, each condition receives a different goal and the comparison is confounded by task difficulty. We verify pairing by asserting that the no-search arm is invariant to K and that no-search and oracle agree exactly at $K = 1$.

We log every candidate’s score at every decision point rather than only the selected index, which permits post-hoc analyses—selection regret, verifier calibration—without rerunning the sweep.

3.8 Exact simulator branching

The oracle arm requires that scoring a candidate leave no trace on the trunk trajectory. We save and restore MuJoCo state using the `mjSTATE_INTEGRATION` specification rather than `mjSTATE_FULLPHYSICS`: the latter omits `qacc_warmstart`, the constraint solver’s cached initial guess, and a differing warmstart causes the solver to converge along a slightly different path. We additionally reapply the warmstart after `mj_forward`, which overwrites it as a side effect, and we roll back mutable Python state on every wrapper in the environment chain—time limits, success latches, cached previous observations—not only the innermost environment. We verify bit-exact restoration under both single and repeated nested branching; the latter is the stricter test, since drift accumulates only across repeated cycles.

4 Search succeeds given a perfect model

Given true dynamics, additional candidates translate into better outcomes on three of six tasks: basketball ($0.62 \rightarrow 0.92$), shelf-place ($0.66 \rightarrow 0.90$), and stick-pull ($0.50 \rightarrow 0.78$). A fourth, bin-picking, improves marginally within noise. This establishes that the candidate sets contain actions materially better than the policy’s first sample, and that the search procedure can find them. Without this control, later negative results would be ambiguous between an ineffective search procedure and an inadequate verifier.

5 With a learned verifier, search is worse than not searching

Our central result is that selecting candidates by a learned verifier degrades performance monotonically in K , and ends below random selection. Pooled across all six tasks, success falls from 0.62 at $K = 1$ to 0.40 at $K = 64$, while random selection remains near 0.58 and the no-search baseline is flat at 0.62 by construction.

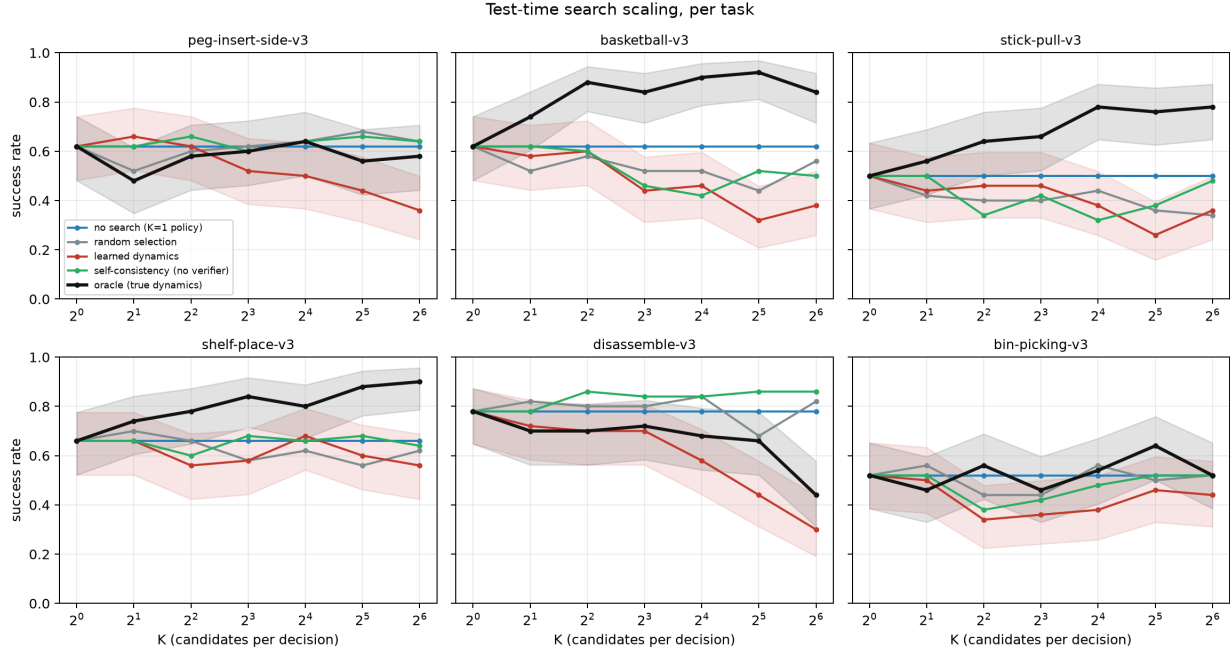


Figure 1: Success rate versus candidate count K , per task, 50 episodes per cell. The oracle arm (black) rises clearly on basketball, stick-pull, and shelf-place. Per-task confidence bands overlap substantially at this sample size; the pooled analysis in Figure 2 carries the inference. Note that the oracle *declines* on disassemble, which we return to in Section 10.

Two features deserve emphasis. First, the random-selection control separates “more candidates is harmful” from “choosing by predicted value is harmful” and supports only the latter. Second, there is no rising phase: degradation begins at $K = 2$ and continues monotonically. This differs from reward-model over-optimization in language models [Gao et al., 2022], where performance typically improves before turning over.

Recent work on test-time scaling for robot policies reports gains rather than degradation. Those evaluations generally use $K \leq 32$ and shorter effective horizons. We note below that per-decision harm in our setting is small ($\sim 5\%$) and accumulates over 15–45 decisions per episode, which offers a plausible reconciliation, though we do not test it directly.

6 The verifier, not the search, is what fails

To separate “no better candidates exist” from “the verifier cannot identify them,” we audit selection directly. At each decision point we score the same candidate pool two ways—with the learned verifier and with true simulator rollouts—and record the true value of the candidate the learned verifier chose, of the best candidate, and of the average candidate.

Normalized regret, $(\text{best} - \text{picked})/(\text{best} - \text{mean})$, sits at approximately 1.0 across the full range of K : the learned verifier’s selection is worth no more than an arbitrary candidate. Better candidates demonstrably appear as K grows, and the verifier finds none of them.

One tension remains unresolved. Regret near 1.0 implies picks that are average *with respect to true short-horizon return*, yet the learned arm’s task success (0.30) falls far below random selection’s (0.82) on this task. The learned verifier therefore appears to select on some feature that is uncorrelated with short-horizon return while being harmful to eventual task completion. We do

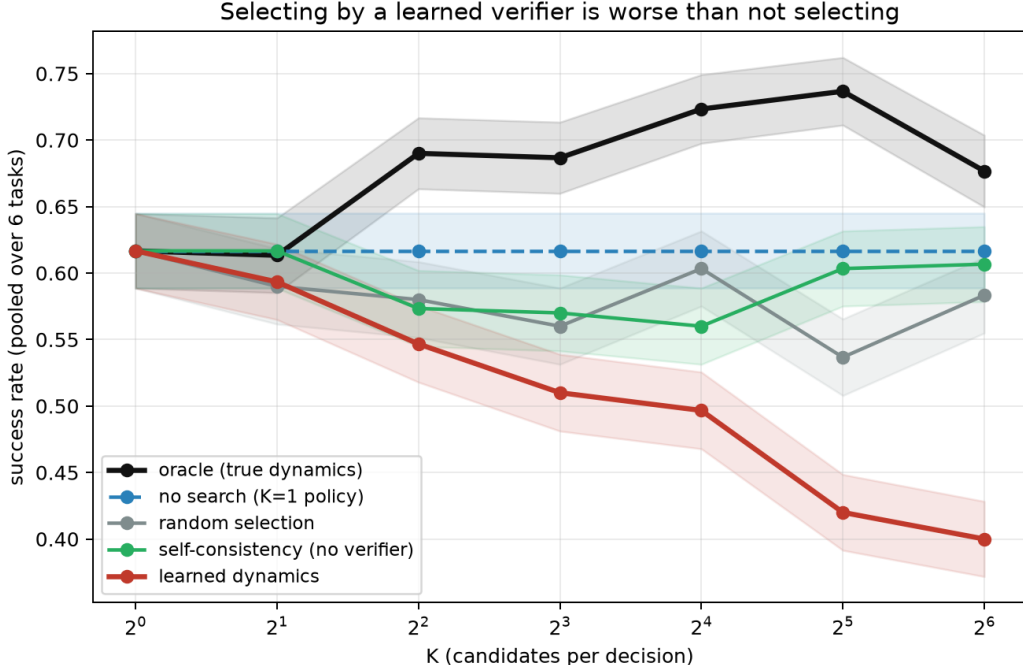


Figure 2: Success rate versus K , pooled over six tasks (300 episodes per cell, shaded bands are ± 1 standard error). Selecting by the learned verifier (red) falls well below both no-search (blue) and random selection (grey), while the oracle (black) rises. The random control rules out candidate count as the cause: the harm comes specifically from selecting on predicted value.

β rank	$K=1$	$K=2$	$K=4$	$K=8$	$K=16$	$K=32$	$K=64$
0 (none)	0.62	0.59	0.55	0.51	0.50	0.42	0.40
1	0.62	0.60	0.54	0.52	0.49	0.41	0.41
2	0.62	0.61	0.58	0.54	0.48	0.41	0.43
3 (largest)	0.62	0.60	0.59	0.53	0.49	0.44	0.41

Table 2: Pooled success by disagreement-penalty strength, with β ranked within each task since absolute values are task-calibrated. Curves are indistinguishable.

not identify that feature.

We also note that the per-decision effect is small—roughly a 5% reduction in success probability relative to an average candidate—while the episode-level effect is a 48-point drop. The mechanism is accumulation across many decisions, not any single catastrophic choice.

7 Uncertainty penalties do not help

A natural remedy is to penalize candidates the model is uncertain about, and it is the standard mitigation in the reward-model literature. Degradation is essentially unchanged across two orders of magnitude in β (Table 2). Ensemble disagreement, at least as computed here, does not identify the candidates that search should avoid.

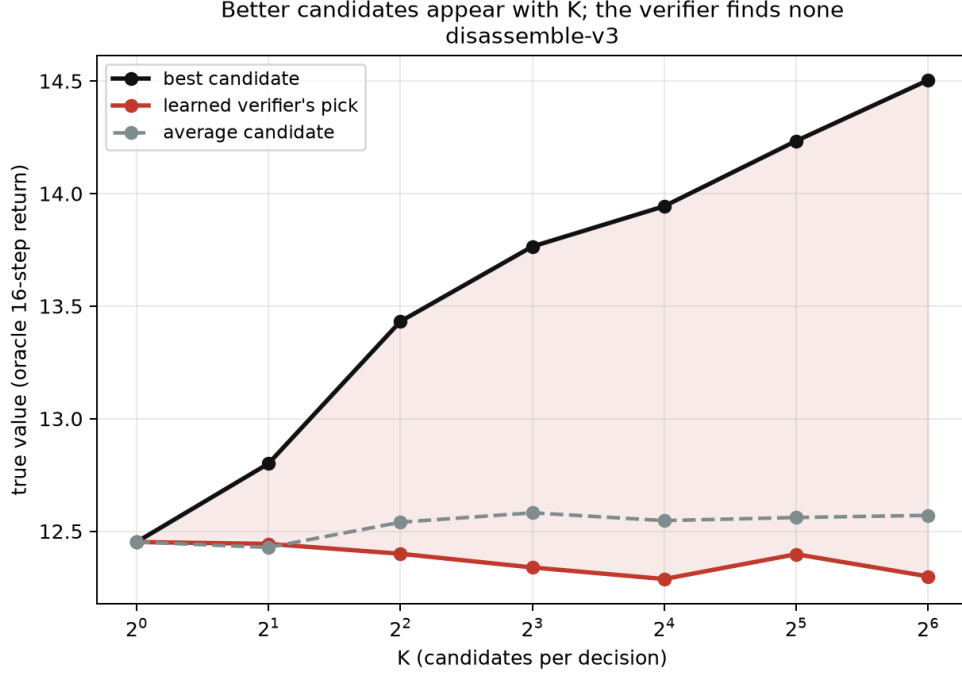


Figure 3: Selection audit on disassemble, 435 decision points. As K grows the best available candidate improves substantially ($12.45 \rightarrow 14.50$ in true 16-step return), while the learned verifier’s pick remains flat at ~ 12.3 , statistically indistinguishable from the average candidate (~ 12.5). Normalized regret is 0.93–1.08 at every K .

8 Model quality: a null result

We further asked whether degradation scales with dynamics-model error. We trained model ladders spanning a $12\times$ range in median 4-step prediction error ($0.19\text{--}2.22\sigma$ on disassemble; $0.11\text{--}4.44\sigma$ on basketball) by jointly reducing training data, network width, and training length, and reran the sweep at each rung. We observed no systematic relationship between model error and degradation. With four rungs per task and a per-condition standard error near 0.07, this experiment cannot distinguish a modest relationship from none, and we report it as a null rather than as evidence of independence.

9 What survives: verifier-free selection

Self-consistency—selecting the candidate closest to all others, using no learned scorer—does not degrade. Pooled success is 0.62 at $K = 1$ and 0.61 at $K = 64$; on disassemble it improves ($0.78 \rightarrow 0.86$). Having no learned proxy, it has no proxy to over-optimize. It is also the cheapest arm, requiring no additional model. Where a verifier of uncertain quality is the only option, this is the safer default.

10 A distinct failure mode: objective misspecification

On disassemble the oracle arm also degrades ($0.78 \rightarrow 0.44$) despite having perfect dynamics, which implicates the scoring objective rather than model error. Three observations are consistent with

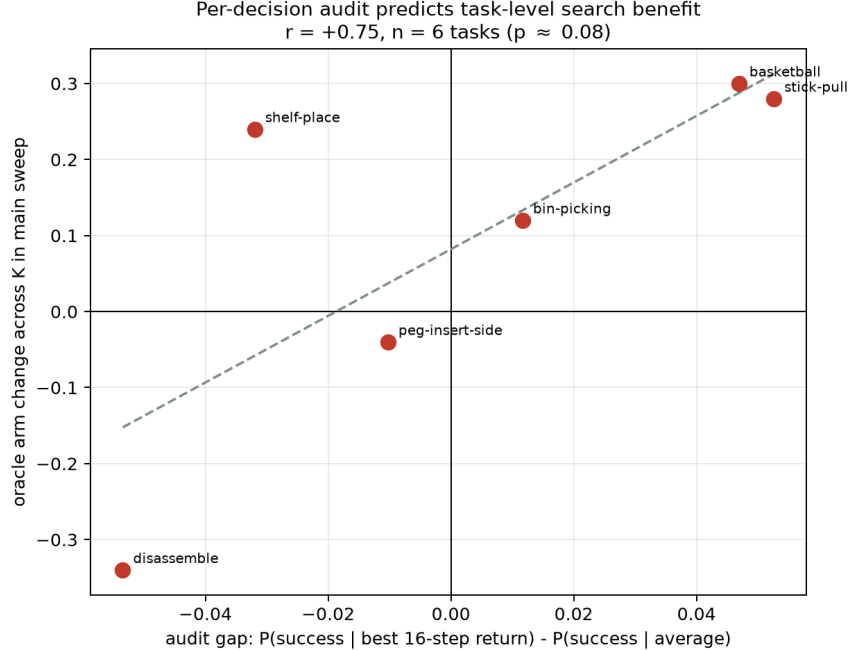


Figure 4: Per-task audit gap—the success advantage of the highest-16-step-return candidate over an average candidate—against the change in oracle-arm success across K in the main sweep. One point per task, $r = +0.75$, $n = 6$ ($p \approx 0.08$). Shelf-place is discordant.

this: the oracle declines; a success-based audit finds that the highest 16-step-return candidate succeeds no more often than an average one; and model quality across a $12\times$ range produces no trend.

The relationship in Figure 4 is suggestive but not established: $n = 6$ tasks, individual audit gaps are within noise at 90 decision points each, and shelf-place has a negative gap despite a strongly positive oracle slope. We present this as a hypothesis the data is consistent with—that maximizing short-horizon return is misaligned with task completion on some tasks, and that search amplifies this misalignment regardless of model quality—rather than as a finding. It is consistent with the objective-mismatch phenomenon identified by [lambert2020objective](#).

11 Limitations

- **One policy class.** Candidates are Gaussian perturbations of a single predicted action chunk rather than samples from a multimodal distribution. Candidate diversity is a plausible moderator we did not vary; a diffusion policy might behave differently.
- **Single seed.** One policy, dynamics model, and value function per task. We have no estimate of training variance.
- **Statistical power.** 50 episodes per cell gives a standard error near 0.07. This is adequate for the 22-point main effect but marginal for every secondary comparison, and per-task curves should be read as context rather than as independent evidence.
- **The oracle is not an upper bound.** It has perfect dynamics but an imperfect objective, and Section 10 shows this matters.

- **Simulation and state observations only.** No pixel observations, no physical robot.

12 Conclusion

Test-time search is not free in control. Across six manipulation tasks, increasing the candidate budget while selecting with a learned verifier monotonically degraded task success, ending below the level achieved by ignoring the verifier entirely and choosing at random. The failure is not in the search: an oracle verifier with true dynamics improved success on half the suite, and a direct audit confirms that better candidates become available as K grows. The failure is that the learned verifier cannot identify them, and that selecting hard on its scores is worse than not selecting at all.

Two practical implications follow. First, verifier quality rather than search budget is the binding constraint, and reporting gains at $K \leq 32$ on short-horizon tasks may not extrapolate—the harm we measure is roughly 5% per decision and only becomes visible when accumulated across dozens of decisions. Second, where verifier quality is uncertain, verifier-free selection by candidate consensus is a safer default: it does not improve much, but it does not degrade, and it requires no additional model.

Two directions seem worth pursuing. The most immediate is candidate diversity: our candidates are perturbations of a single mode, and it is an open question whether a genuinely multimodal proposal distribution changes the picture, either by offering the verifier a better-separated choice or by widening the space in which it can go wrong. The second concerns what the learned verifier is actually selecting on. Our audit shows its picks are average with respect to true short-horizon return yet substantially harmful to task completion, which implies selection on some feature orthogonal to the scoring objective. Identifying that feature would say more about why learned verifiers fail than another sweep over K .

References

- [Chi et al., 2023] C. Chi, S. Feng, Y. Du, Z. Xu, E. Cousineau, B. Burchfiel, and S. Song. Diffusion policy: Visuomotor policy learning via action diffusion. In *Robotics: Science and Systems*, 2023.
- [Chua et al., 2018] K. Chua, R. Calandra, R. McAllister, and S. Levine. Deep reinforcement learning in a handful of trials using probabilistic dynamics models. In *Advances in Neural Information Processing Systems*, 2018.
- [Gao et al., 2022] L. Gao, J. Schulman, and J. Hilton. Scaling laws for reward model overoptimization. *arXiv preprint arXiv:2210.10760*, 2022.
- [Hansen et al., 2023] N. Hansen, H. Su, and X. Wang. TD-MPC2: Scalable, robust world models for continuous control. *arXiv preprint arXiv:2310.16828*, 2023.
- [Janner et al., 2019] M. Janner, J. Fu, M. Zhang, and S. Levine. When to trust your model: Model-based policy optimization. In *Advances in Neural Information Processing Systems*, 2019.
- [Lambert et al., 2020] N. Lambert, B. Amos, O. Yadan, and R. Calandra. Objective mismatch in model-based reinforcement learning. In *Learning for Dynamics and Control*, 2020.

- [Snell et al., 2024] C. Snell, J. Lee, K. Xu, and A. Kumar. Scaling LLM test-time compute optimally can be more effective than scaling model parameters. *arXiv preprint arXiv:2408.03314*, 2024.
- [RoboMonkey, 2025] RoboMonkey: Scaling test-time sampling and verification for vision-language-action models. In *Conference on Robot Learning*, 2025.
- [RoVer, 2025] RoVer: Test-time scaling for robot manipulation with process reward models. *arXiv preprint arXiv:2510.10975*, 2025.
- [ELASTIC, 2026] ELASTIC: Adaptive allocation of test-time compute for robot policies. *arXiv preprint arXiv:2606.31132*, 2026.
- [CoVer-VLA, 2026] CoVer-VLA: Test-time scaling laws for embodied instruction following. *arXiv preprint arXiv:2602.12281*, 2026.
- [Zhao et al., 2023] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn. Learning fine-grained bimanual manipulation with low-cost hardware. In *Robotics: Science and Systems*, 2023.